

计算机体系结构

11. 指令级并行性 II

李建华

计算机与信息学院

The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburgh, and eth.
The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.

本节提纲

- 指令级并行的概念
- 循环展开和指令调度
- **指令的动态调度**
- 分支预测技术简介
- 多指令流出技术

指令的动态调度

- 静态调度

- 依靠编译器对代码进行调度，以减少相关引起的冒险；
- 它**不是在程序执行的过程中**，而是在**编译期间**进行代码调度和优化；
- 之前的**指令调度**和**循环展开**，都是静态调度。

- 动态调度

- 在程序的执行过程中，依靠**专门的硬件**对指令进行调度，以减少数据相关导致的流水线停顿。

指令的动态调度

- 动态调度的优点：
 - 能够处理一些在编译时情况不明的相关，可以减轻编译器的负担；
 - 例如：涉及到存储器访问的相关【访存地址是在**执行段**进行计算的，**编译期间不知道具体的访存地址。**】
 - 能够使本来是面向某一流水线优化编译的代码在其他的流水线（动态调度）上也能高效地执行。
- 动态调度的缺点：
 - 实现动态调度的硬件开销会显著增加
 - 复杂的控制、更多的通路和存储开销

指令的动态调度

- 前面，我们所讨论的流水线的最大的局限性：
 - 指令必须按序流出（进入流水线）和执行，也就是：In-Order Execution（顺序执行）
 - 考虑下面的代码段：

DIV.D **F4**, F0, F2

SUB.D F10, **F4**, F6

ADD.D F12, F6, F14

- SUB.D指令与DIV.D指令关于F4相关，导致流水线停顿；
- **ADD.D指令与前面的指令都没有关系，但其处理也因此受阻；**
- **Q: 有没有方法来突破这里的限制，继续提升流水线性能？**

指令的动态调度

- 在前面的基本流水线中：

一个更好的执行模型就是乱序执行：

Out-of-Order Execution

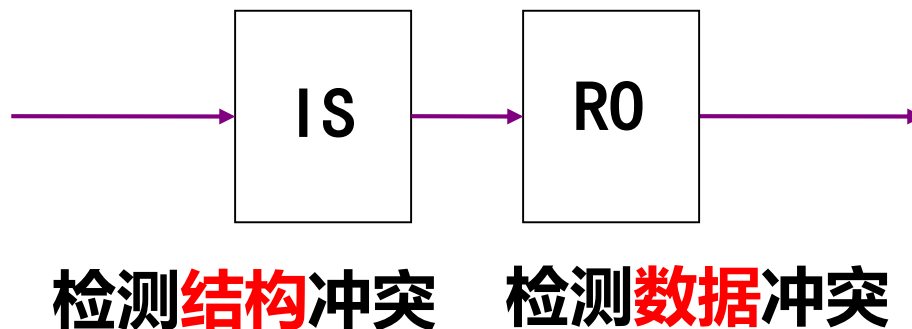
检测结构冲突造成的流水线冒险

检测数据相关造成的流水线冒险

- 一旦一条指令应导致冒险受阻，该指令后续的指令都将停顿，这种处理策略会限制流水线的性能（是性能瓶颈所在）。

指令的动态调度

- 为了突破限制支持乱序执行，可以将5段流水线的**译码段**进行改造，分割为两个阶段：
 - **流出 (Issue, IS, 又叫发射)**：指令译码，检查是否存在结构冲突。(in-order issue)
 - **读操作数 (Read Operands, RO)**：等待数据冲突消失，然后读操作数。



哪条指令先获得数据（值），就可以先行起飞。。。。。

指令的动态调度

- 在顺序执行的MIPS 5-段流水线中，不会发生WAR和WAW造成的冒险。**为什么？**
- 但是，乱序执行就使得WAR和WAW可能造成流水线冒险。

– 例如，考虑下面的代码：

	DIV.D	F10, F0, F2	} 存在输出相关
存在反相关	{ SUB.D	F10, F4, F6	
	ADD.D	F6, F8, F14	

- Tomasulo算法可以通过使用**寄存器重命名**来**动态**消除上面的相关以减少流水线冒险造成的停顿。

动态调度与异常

- I/O device request
- Invoking an operating system service from a user program
- Tracing instruction execution
- Breakpoint (programmer-requested interrupt)
- Integer arithmetic overflow
- FP arithmetic anomaly
- Page fault (not in main memory)
- Misaligned memory accesses (if alignment is required)
- Memory protection violation
- Using an undefined or unimplemented instruction
- Hardware malfunctions
- Power failure

精确与不精确异常

- 不精确异常：
 - 当执行指令*i*导致发生异常时处理机的现场（状态）与 **严格按程序顺序执行指令*i*发生异常时的现场**不同。
 - 不精确异常使得在异常处理后难以接着继续执行程序。 **为什么？**
- 精确异常：
 - 如果发生异常时处理机的现场跟**严格按程序顺序执行指令*i*发生异常时的现场**相同。
 - **思考：**在动态调度下，如何保持精确异常？

动态调度与异常

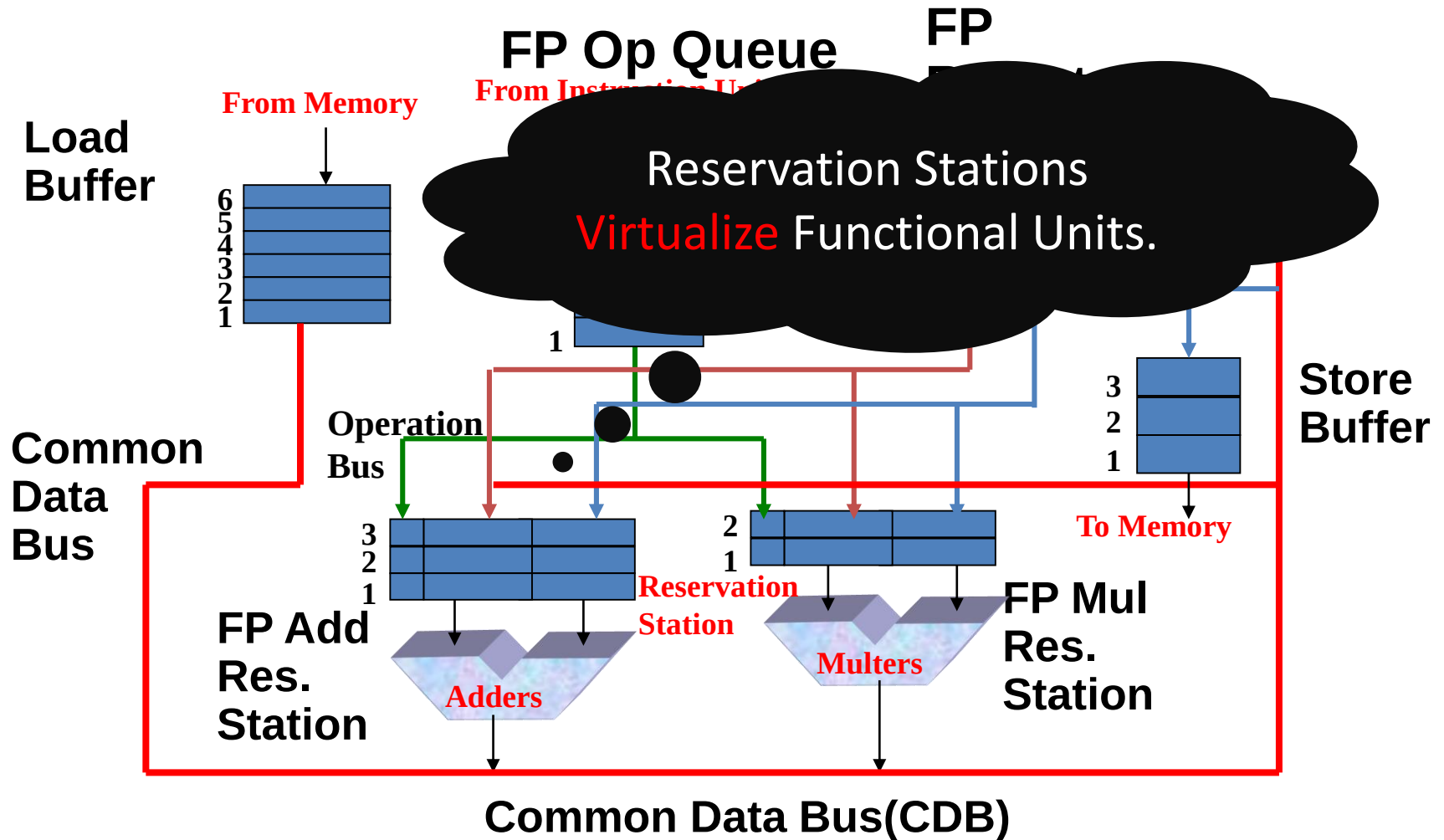
- 动态调度使得指令乱序完成，会增加异常处理的难度；
- 要确保程序结果正确，动态调度需要保持正确的异常行为（另外一个需要保持的是什么？）
 - 对于一条会产生异常的指令来说，只有当处理器**确切地知道**该指令将被执行后，才允许它产生异常。
 - 即使保持了正确的异常行为，动态调度处理机仍可能发生不精确异常。为什么？
 - 在动态调度下，指令乱序执行。当指令 i 发生异常时：
 - 流水线**可能**已经执行完**程序顺序下位于指令 i 之后**的指令；
 - 流水线**可能**还没完成按**程序顺序下位于指令 i 之前**的指令。

Tomasulo算法

- 为IBM 360/91设计的、在CDC 6600三年之后发表(1966)
- 目标：即使在**没有特殊编译支持**的情况下，也能让流水线处理器取得高性能。
- 为什么要学习Tomasulo算法？
 - 深刻地影响了后续一系列产品的设计：Alpha 21264、HP 8000、MIPS 10000、Pentium II、PowerPC 604, ...

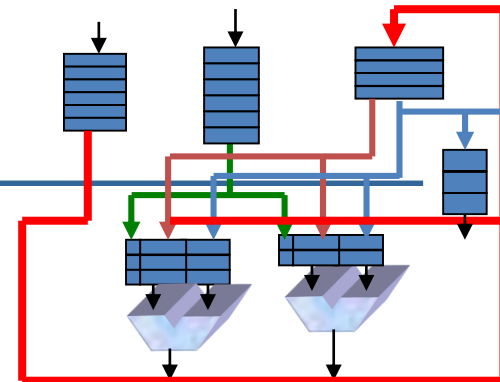
请同学们学习[scoreboard](#)，不理解的地方来找我讨论。

Tomasulo算法架构图



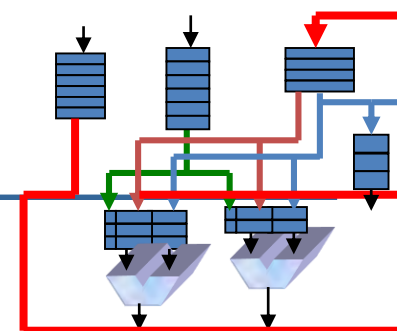
虚拟化有什么作用?

Tomasulo算法的特征



- 控制&缓冲器**分布于**功能部件
 - 功能部件配置了**缓冲器**，称为 “**保留站** (reservation stations)” ，用来**存放未决的操作数（值）**；
- 指令中的**寄存器**被动态地换成**数值**或者**指向保留站的指针**，这一过程称为：**寄存器重命名(register renaming)**
 - 消除WAR、WAW相关 (名相关) 造成的流水线冒险；
 - 保留站比寄存器多，可完成编译阶段不能完成的一些优化工作；
- 计算结果从RS通过公共数据总线（Common Data Bus）直通FU，无需通过寄存器；
- 装入（Load）和 存储（Stores）也象其他功能部件一样具有保留站（专门的缓冲器）；

Tomasulo算法的三个阶段



1. Issue—从FP Op Queue中取出指令

- 如果保留站空闲 (无结构冒险), 控制机制发射指令&发送操作数(对寄存器进行换名, 消除名相关)。

2. Execution—对操作数执行操作(EX)

- 如果**两个操作数都已就绪**, 就执行; (RAW的处理)
- 如果没有就绪, 就**观测**公共数据总线**等待**所需结果;

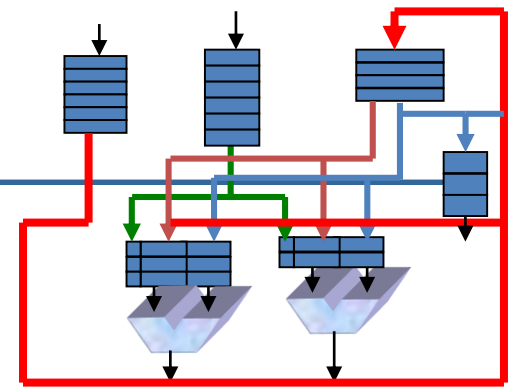
3. Write result—完成执行(WB)

- 通过公共数据总线将结果写入到所有等待的部件;
- 标记保留站可用 (Not busy);

• 公共数据总线: 数据 + 源 (“来源” 总线)

- 64位数据 (Value) + 4位功能部件编码 (指令相关的纽带);
- 如果与期望的功能部件匹配, 就 “写” (产生结果);
 - 在CDB上**广播** (broadcast)上面的结果;

保留站的组成



- **Op**—该部件将完成的具体操作
- **Vj, Vk**—源操作数的实际数值
 - 存储缓冲器(Store buffers)设有V域, 存放将存储的结果;
- **Qj, Qk**—将产生源寄存器值 (将写的值) 的保留站
- **Busy**—指明 RS 或 FU 处于忙状态
- **Register result status**—指明哪个功能部件将写到哪个寄存器(Qi)。如果没有将写入寄存器的未决指令, 该域为空;

Tomasulo示例-第0周期

<u>Instruction status</u>				<i>Execution Write</i>		
Instruction	<i>j</i>	<i>k</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>	Busy Address
LD F6	34+	R2				Load1 No
LD F2	45+	R3				Load2 No
MUL F0	F2	F4				Load3 No
SUB F8	F6	F2				
DIV F10	F0	F6				
ADD F6	F8	F2				

<u>Reservation Stations</u>			$S1$	$S2$	$RS\ for\ j$	$RS\ for\ k$
$Time$	$Name$	$Busy\ Op$	V_j	V_k	Q_j	Q_k
0	Add1	No				
0	Add2	No				
0	Add3	No				
0	Mult1	No				
0	Mult2	No				

LD:	2 cycles
ADD:	2 cycles
Mult:	10 cycles
Divd:	40 cycles

Register result status

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>	
0	<i>FU</i>										

Tomasulo示例-第1周期

Instruction status *Execution Write*

Instruction *j* *k* *Issue* *complete* *Result*

LD F6 34+ R2

LD F2 45+ R3

MUL F0 F2 F4

SUB F8 F6 F2

DIV F10 F0 F6

ADD F6 F8 F2

1

2 Load1 *Busy* *Address*

0 Load2 No 34+R2

0 Load3 No

Reservation Stations

S1

S2

RS for j

RS for k

Time *Name* *Busy* *Op*

Vj

Vk

Qj

Qk

0 Add1 No

0 Add2 No

Add3 No

0 Mult1 No

0 Mult2 No

Register result status

Clock

F0

F2

F4

F6

F8

F10

F12 ...

F30

1

FU

Load1

Tomasulo示例-第2周期

Instruction status *Execution Write*

Instruction *j* *k* *Issue* *complete* *Result*

LD F6 34+ R2

LD F2 45+ R3

MUL F0 F2 F4

SUB F8 F6 F2

DIV F10 F0 F6

ADD F6 F8 F2

Busy Address

1 Load1 Yes 34+R2

2 Load2 Yes 45+R3

0 Load3 No

Reservation Stations

S1

S2

RS for j

RS for k

Time Name Bus Op

Vj

Vk

Qj

Qk

0 Add1 No

0 Add2 No

Add3 No

0 Mult1 No

0 Mult2 No

Register result status

Clock

F0

F2

F4

F6

F8

F10 F12 ...

F30

2

FU

Load2

Load1

Tomasulo示例-第3周期

Instruction status				Execution		Write			
Instruction	<i>j</i>	<i>k</i>		Issue	complete	Result		Busy	Address
LD F6	34+	R2		1	3		0 Load1	Yes	34+R2
LD F2	45+	R3		2			1 Load2	Yes	45+R3
MUL F0	F2	F4		3			0 Load3	No	
SUB F8	F6	F2							
DIV F10	F0	F6							
ADD F6	F8	F2							

Reservation Stations				<i>S1</i>	<i>S2</i>	<i>RS for j</i>	<i>RS for k</i>
Time	Name	Bus	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	Yes	MULTD		R(F4)	Load2	
0	Mult2	No					

Register result status										
Clock		<i>F0</i>	<i>F2</i>		<i>F4</i>		<i>F6</i>		<i>F8</i>	<i>F10 F12 ... F30</i>
3	FU	Mult1	Load2				Load1			

- 保留站中寄存器名被“换名”； F2 -> Load2
- Load1完成，哪些指令在等待Load1的结果？

Tomasulo示例-第4周期

Instruction status				Execution Write				
Instruction	j	k		Issue	complete	Result	Busy	Address
LD F6	34+	R2		1	3	4	0 Load1	No
LD F2	45+	R3		2	4		0 Load2	Yes 45+R3
MUL F0	F2	F4		3			0 Load3	No
SUB F8	F6	F2		4				
DIV F10	F0	F6						
ADD F6	F8	F2						

Reservation Stations			S1	S2	RS for j	RS for k
Time	Name	Bus Op	Vj	Vk	Qj	Qk
0	Add1	Yes SUB	M(34+R2)			Load2
0	Add2	No				
	Add3	No				
0	Mult1	Yes MULT	D	R(F4)	Load2	
0	Mult2	No				

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12...	F30
4	FU	Mult1	Load2		M(34+R2)	Add1			

- Load2将完成，哪些指令在等待Load2?

Tomasulo示例-第5周期

Instruction status				Execution Write				
Instruction	<i>j</i>	<i>k</i>		Issue	complete	Result	Busy	Address
LD F6	34+	R2		1	3	4	Load1	No
LD F2	45+	R3		2	4	5	Load2	No
MUL F0	F2	F4		3			Load3	No
SUB F8	F6	F2		4				
DIV F10	F0	F6		5				
ADD F6	F8	F2						

Reservation Stations			S1	S2	RS for <i>j</i>	RS for <i>k</i>
Time	Name	Bus Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
2	Add1	Yes	SUBC	M(34+R2)	M(45+R3)	
0	Add2	No				
	Add3	No				
10	Mult1	Yes	MULT	M(45+R3)	R(F4)	
0	Mult2	Yes	DIVD		M(34+R2)	Mult1

SUBD与MULT同时从CDB获得值。

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12...	F30
5	FU	Mult1	M(45+R3)		M(34+R2)	Add1	Mult2		

Tomasulo示例-第6周期

Instruction status				Execution Write				
Instruction	<i>j</i>	<i>k</i>		Issue	complete	Result	Busy	Address
LD F6	34+	R2		1	3	4	Load1	No
LD F2	45+	R3		2	4	5	Load2	No
MUL F0	F2	F4		3			Load3	No
SUB F8	F6	F2		4				
DIV F10	F0	F6		5				
ADD F6	F8	F2		6				

Reservation Stations			S1	S2	RS for <i>j</i>	RS for <i>k</i>
Time	Name	Busy	Op	<i>V_j</i>	<i>V_k</i>	<i>Q_j</i>
1	Add1	Yes	SUB	M(34+R2)	M(45+R3)	
0	Add2	Yes	ADD		M(45+R3)	Add1
	Add3	No				
9	Mult1	Yes	MULT	M(45+R3)	R(F4)	
0	Mult2	Yes	DIV		M(34+R2)	Mult1

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
6	FU	Mult1	M(45+R3)		Add2	Add1	Mult2			

- 发射ADDD

Tomasulo示例-第7周期

Instruction status				Execution Write				
Instruction	j	k		Issue	complete	Result	Busy	Address
LD F6 34+ R2				1	3	4	Load1	No
LD F2 45+ R3				2	4	5	Load2	No
MUL F0 F2 F4				3			Load3	No
SUB F8 F6 F2				4	7			
DIV F10 F0 F6				5				
ADD F6 F8 F2				6				

Reservation Stations				S1	S2	RS for j	RS for k
Time	Name	Busy	Op	Vj	Vk	Qj	Qk
0	Add1	Yes	SUBD	M(34+R2)	M(45+R3)		
0	Add2	Yes	ADD		M(45+R3)	Add1	
	Add3	No					
8	Mult1	Yes	MULT	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
7	FU	Mult1	M(45+R3)		Add2	Add1	Mult2			

- Add1完成, 哪些指令在等待Add1?

Tomasulo示例-第8周期

Instruction status				Execution Write				
Instruction	<i>j</i>	<i>k</i>		Issue	complete	Result	Busy	Address
LD F6	34+	R2		1	3	4	Load1	No
LD F2	45+	R3		2	4	5	Load2	No
MUL F0	F2	F4		3			Load3	No
SUB F8	F6	F2		4	7	8		
DIV F10	F0	F6		5				
ADD F6	F8	F2		6				

Reservation Stations				S1	S2	RS for <i>j</i>	RS for <i>k</i>
Time	Name	Busy	Op	<i>V_j</i>	<i>V_k</i>	<i>Q_j</i>	<i>Q_k</i>
0	Add1	No					
2	Add2	Yes	ADDC	M()-M()	M(45+R3)		
	Add3	No					
7	Mult1	Yes	MULT	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)		Mult1

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12...	F30
8	FU	Mult1	M(45+R3)		Add2	M()-M()	Mult2		

Tomasulo示例-第9周期

Instruction	j	k	Issue	complete	Result	Busy	Address
LD	F6	34+	R2	1	3	4	Load1
LD	F2	45+	R3	2	4	5	Load2
MULT	F2	F4	F4	3			Load3
SUB	F8	F6	F2	4	7	8	
DIV	F10	F0	F6	5			
ADD	F6	F8	F2	6			

Reservation Stations			S1	S2	RS for j	RS for k	
Time	Name	Busy	Op	Vj	Vk	Qj	Qk
0	Add1	No					
1	Add2	Yes	ADD	M()-M()	M(45+R3)		
	Add3	No					
6	Mult1	Yes	MULT	M(45+R3)	R(F4)		
0	Mult2	Yes	DIV		M(34+R2)	Mult1	

Register result status

Clock	F0	F2	F4	F6	F8	F10	F12...	F30
9	FU	Mult1	M(45+R3)		Add2	M()-M()	Mult2	

Tomasulo示例-第10周期

<u>Instruction status</u>				<u>Execution Write</u>				
Instruction	<i>j</i>	<i>k</i>		Issue	complete	Result	Busy	Address
LD F6	34+	R2		1	3	4	Load1	No
LD F2	45+	R3		2	4	5	Load2	No
MUL F0	F2	F4		3			Load3	No
SUB F8	F6	F2		4	7	8		
DIV F10	F0	F6		5				
ADD F6	F8	F2		6	10			

<u>Reservation Stations</u>				<i>S1</i>	<i>S2</i>	<i>RS for j</i>	<i>RS for k</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
0	Add1	No					
0	Add2	Yes	ADDC	M()-M()	M(45+R3)		
	Add3	No					
5	Mult1	Yes	MULT	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

<u>Register result status</u>		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i> ...	<i>F30</i>
Clock									
10	FU	Mult1	M(45+R3)		Add2	M0-M0	Mult2		

- Add2完成，哪些指令在等待Add2?

Tomasulo示例-第11周期

<u>Instruction status</u>				<u>Execution Write</u>				
Instruction	<i>j</i>	<i>k</i>		Issue	complete	Result	Busy	Address
LD F6	34+	R2		1	3	4	Load1	No
LD F2	45+	R3		2	4	5	Load2	No
MUL F0	F2	F4		3			Load3	No
SUB F8	F6	F2		4	7	8		
DIV F10	F0	F6		5				
ADD F6	F8	F2		6	10	11		

<u>Reservation Stations</u>			<i>S1</i>	<i>S2</i>	<i>RS for j</i>	<i>RS for k</i>
Time	Name	Bus Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
0	Add1	No				
0	Add2	No				
	Add3	No				
4	Mult1	Yes	MULT	M(45+R3)	R(F4)	
0	Mult2	Yes	DIVD	M(34+R2)	Mult1	

Register result status

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12 ...</i>	<i>F30</i>
11	FU	Mult1	M(45+R3)		(M-M)+M()	M()-M()	Mult2		

- ADDD在该周期写结果

Tomasulo示例-第12周期

Instruction status				Execution Write				
Instruction	j	k		Issue	complete	Result	Busy	Address
LD F6 34+ R2				1	3	4	Load1	No
LD F2 45+ R3				2	4	5	Load2	No
MUL F0 F2 F4				3			Load3	No
SUB F8 F6 F2				4	6	7		
DIV F10 F0 F6				5				
ADD F6 F8 F2				6	10	11		

Reservation Stations			S1	S2	RS for j	RS for k
Time	Name	Bus Op	Vj	Vk	Qj	Qk
0	Add1	No				
0	Add2	No				
	Add3	No				
3	Mult1	Yes	MULT	M(45+R3)	R(F4)	
0	Mult2	Yes	DIVD	M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12 ...	F30
12	FU	Mult1	M(45+R3)		(M-M)+M()	M0-M0	Mult2		

- 注意：所有短周期指令都已经完成

Tomasulo示例-第13周期

Instruction status				Execution Write				
Instruction	j	k		Issue	complete	Result	Busy	Address
LD F6	34+	R2		1	3	4	Load1	No
LD F2	45+	R3		2	4	5	Load2	No
MUL F0	F2	F4		3			Load3	No
SUB F8	F6	F2		4	7	8		
DIV F10	F0	F6		5				
ADD F6	F8	F2		6	10	11		

Reservation Stations				S1	S2	RS for j	RS for k
Time	Name	Bus	Op	Vj	Vk	Qj	Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
2	Mult1	Yes	MULT	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12...	F30
13	FU	Mult1	M(45+R3)		(M-M)+M()	M0-M0	Mult2		

Tomasulo示例-第14周期

Instruction status				Execution Write				
Instruction	<i>j</i>	<i>k</i>		Issue complete	Result		Busy	Address
LD F6	34+	R2		1	3	4	Load1	No
LD F2	45+	R3		2	4	5	Load2	No
MUL F0	F2	F4		3			Load3	No
SUB F8	F6	F2		4	7	8		
DIV F10	F0	F6		5				
ADD F6	F8	F2		6	10	11		

<u>Reservation Stations</u>			$S1$	$S2$	$RS \text{ for } j$	$RS \text{ for } k$
$Time$	$Name$	Bus_Op	V_j	V_k	Q_j	Q_k
0	Add1	No				
0	Add2	No				
	Add3	No				
1	Mult1	Yes	MULT	$M(45+R3)$	$R(F4)$	
0	Mult2	Yes	DIVD		$M(34+R2)$	Mult1

Register result status		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12 \dots$	$F30$
Clock									
14	FU	Mult1	$M(45+R3)$		$(M-M)+M()$	$M()-M()$	Mult2		

Tomasulo示例-第15周期

Instruction status				Execution Write				
Instruction	j	k		Issue	complete	Result	Busy	Address
LD F6 34+ R2				1	3	4	Load1	No
LD F2 45+ R3				2	4	5	Load2	No
MUL F0 F2 F4				3	15		Load3	No
SUB F8 F6 F2				4	7	8		
DIV F10 F0 F6				5				
ADD F6 F8 F2				6	10	11		

<u>Reservation Stations</u>				S1	S2	RS for j	RS for k
Time	Name	Busy	Op	Vj	Vk	Qj	Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	Yes	MULT	M(45+R3)	R(F4)		
0	Mult2	Yes	DIVD		M(34+R2)	Mult1	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
15	FU	Mult1	M(45+R3)		(M-M)+M()	M()-M()	Mult2			

- Mult1 完成，哪些指令在等待Mult1？

Tomasulo示例-第16周期

Instruction status				Execution Write				
Instruction	<i>j</i>	<i>k</i>		Issue	complete	Result	Busy	Address
LD F6	34+	R2		1	3	4	Load1	No
LD F2	45+	R3		2	4	5	Load2	No
MUL F0	F2	F4		3	15	16	Load3	No
SUB F8	F6	F2		4	7	8		
DIV F10	F0	F6		5				
ADD F6	F8	F2		6	10	11		

Reservation Stations			S1	S2	RS for <i>j</i>	RS for <i>k</i>
Time	Name	Busy	Op	V _j	V _k	Q _j Q _k
0	Add1	No				
0	Add2	No				
	Add3	No				
0	Mult1	No				
40	Mult2	Yes	DIVD	M*F4	M(34+R2)	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
16	FU	M*F4	M(45+R3)		(M-M)+M()	M0-M0	Mult2			

- 注意: 只有除法指令没有完成

Tomasulo示例-第55周期

Instruction status				Execution Write				
Instruction	<i>j</i>	<i>k</i>		Issue	complete	Result	Busy	Address
LD F6	34+	R2		1	3	4	Load1	No
LD F2	45+	R3		2	4	5	Load2	No
MUL F0	F2	F4		3	15	16	Load3	No
SUB F8	F6	F2		4	7	8		
DIV F10	F0	F6		5				
ADD F6	F8	F2		6	10	11		

Reservation Stations			S1	S2	RS for <i>j</i>	RS for <i>k</i>
Time	Name	Busy	Op	V _j	V _k	Q _j Q _k
0	Add1	No				
0	Add2	No				
	Add3	No				
0	Mult1	No				
1	Mult2	Yes	DIVD	M*F4	M(34+R2)	

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
55	FU	M*F4	M(45+R3)		(M-M)+M()	M()-M()	Mult2			

Tomasulo示例-第56周期

Instruction status				Execution Write				
Instruction	j	k		Issue	complete	Result	Busy	Address
LD F6	34+	R2		1	3	4	Load1	No
LD F2	45+	R3		2	4	5	Load2	No
MUL F0	F2	F4		3	15	16	Load3	No
SUB F8	F6	F2		4	7	8		
DIV F10	F0	F6		5	56			
ADD F6	F8	F2		6	10	11		

<u>Reservation Stations</u>			<i>S1</i>	<i>S2</i>	<i>RS for j</i>	<i>RS for k</i>	
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	No					
0	Mult2	Yes	DIVD	M*F4	M(34+R2)		

Register result status

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
56	FU	M*F4	M(45+R3)		(M-M)+M()	M()-M()	Mult2			

- Mult2 完成

Tomasulo示例-第57周期

Instruction status				Execution Write				
Instruction	j	k		Issue	complete	Result	Busy	Address
LD F6	34+	R2		1	3	4	Load1	No
LD F2	45+	R3		2	4	5	Load2	No
MUL F0	F2	F4		3	15	16	Load3	No
SUB F8	F6	F2		4	7	8		
DIV F10	F0	F6		5	56	57		
ADD F6	F8	F2		6	10	11		

Reservation Stations				S1	S2	RS for j	RS for k
Time	Name	Busy	Op	Vj	Vk	Qj	Qk
0	Add1	No					
0	Add2	No					
	Add3	No					
0	Mult1	No					
0	Mult2	No					

Register result status		F0	F2	F4	F6	F8	F10	F12	...	F30
Clock										
57	FU	M*F4	M(45+R3)		(M-M)+M()	M0-M0	M*F4/M			

- 在基于tomasulo算法的动态调度中,指令是：**按序发送、乱序执行、乱序完成的。**
- Q: Tomasulo能否实现精确异常？**

Tomasulo算法的缺点

- 功能实现复杂，控制复杂
 - 需要复杂的硬件实现相应的功能
- 需要大量高速的相联存储，存储开销大；
- CDB是制约性能增长的瓶颈
 - 每个CDB必须广播到多个功能部件单元 $\Rightarrow$ 大容量、写操作密集；
 - 每个周期可以同时完成的功能部件数量可能由于**单总线**而受限（最坏情况为“1”）！
 - 多个CDB $\Rightarrow$ 为完成并行相联存储，功能部件FU需要更复杂的逻辑控制；

Tomasulo对于龙芯杯团体赛是基础操作！

本节提纲

- 指令级并行的概念
- 循环展开和指令调度
- 指令的动态调度
- **分支预测技术简介**
- 多指令流出技术

动态分支预测技术

- 动态分支预测：在程序运行时（runtime），根据分支指令**过去的历史**来**预测其将来的行为**。
 - 如果分支行为发生了变化，预测结果也跟着改变；
 - **相比于静态预测，有更好的预测准确度和适应性。**
- 分支预测带来的性能提升取决于：
 - 预测的准确性 — 正确率；
 - 预测正确以及不正确时的开销。
- 决定分支预测开销的因素：
 - 流水线的结构及预测方法的开销；
 - 预测错误时的恢复策略的开销。

动态分支预测技术

- 动态分支预测技术的三个任务：
 - ① 预测**是否是**分支指令
 - ② 预测分支**是否**成功 (跳转)
 - ③ 预测分支的目标地址
- 对于任务②，需要解决二个关键问题：
 - 如何记录**分支成功与否**的历史信息？
 - 如何根据历史信息来预测分支？[**预测的策略**]
- 动态分支预测技术的**后勤保障**：
 - 预测失败时，如何保持**正确性红线**？[**Keep in mind**]

动态分支预测技术后勤保障

- 在预测错误时，必须具备作废已经取指并进入后续流水段处理的指令，恢复现场，并从正确的分支路径重新取指令（的能力）。



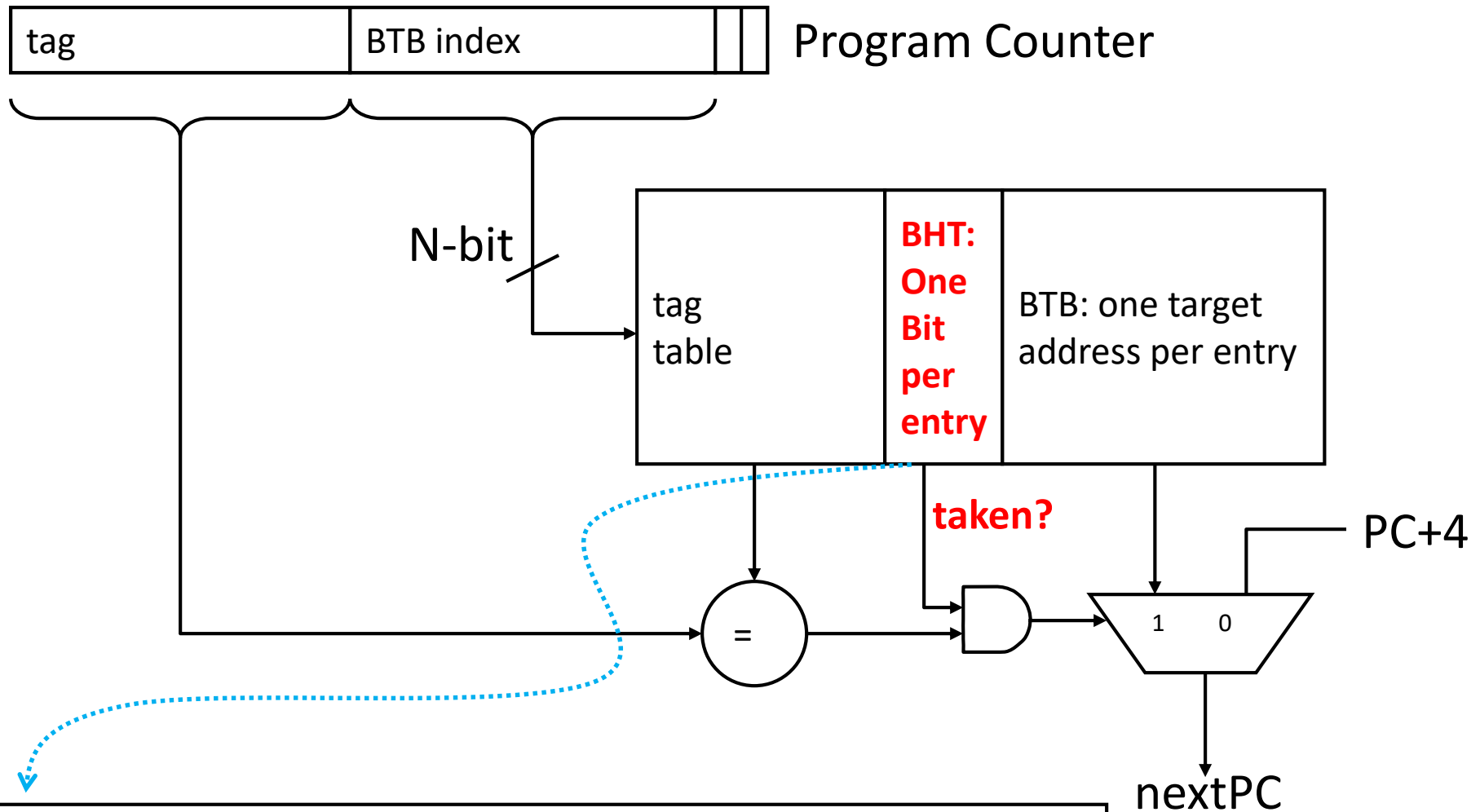
图片来自[这里](#)

你可以在课设里面大展身手！

预测跳转：分支历史表

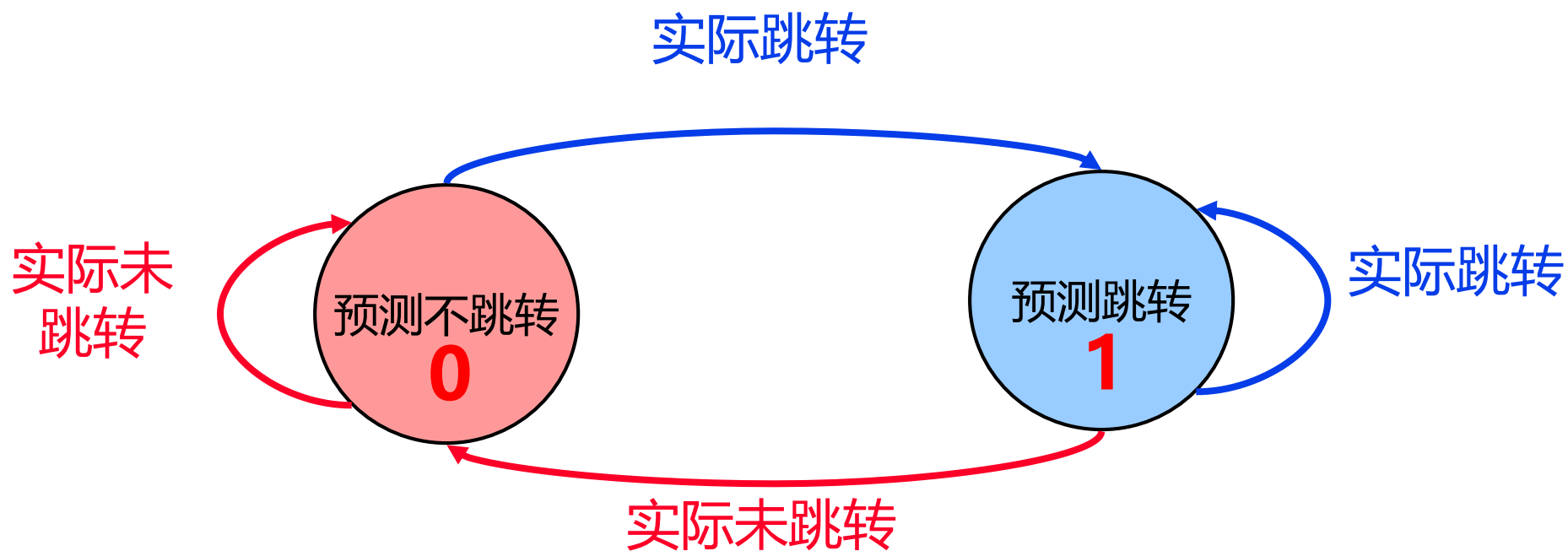
- BHT (Branch History Table) 或 (Branch Prediction Buffer)
 - 最简单的动态分支预测方法；
 - 用BHT来记录分支指令最近一次或几次的执行情况（成功或不成功），并据此进行预测；
- Naïve Method：只有1个预测位的分支预测缓冲
 - 记录分支指令最近一次的历史，BHT中只需要**1位二进制位 + PC或者PC的部分位**。
 - 该方法又叫**Last-Time Predictor**，其优点是：简单，缺点是：预测的精度并不高。

Last-Time Predictor的实现



The 1-bit BHT entry is updated with the correct outcome after each execution of a branch

预测跳转的状态机



该方案预测的精度并不高

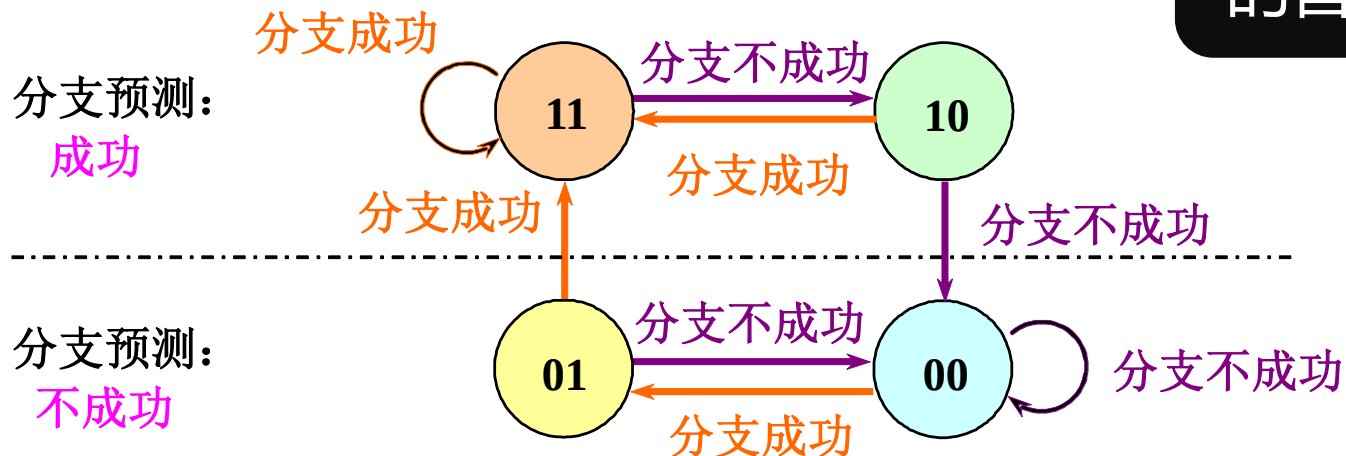
优化Last-Time Predictor

- **问题:** Last-Time Predictor 改变预测结果 ($T \rightarrow NT$ or $NT \rightarrow T$) 太快了
 - 即使对于绝大多数情况下T或者NT的分支来说, 也是如此。
- **解决方法:** 为预测器添加滞后性, 让其在面临一次预测错误时不至于立即改变预测。
 - 使用2个bits来记录分支指令的历史
 - 这样处理后, T 与 NT 均具有2个状态。

Smith, “[A Study of Branch Prediction Strategies](#),” ISCA 1981.

Bimodal Predictor

- **优化：** 用更多比特来记录历史，可以提高精度。
 - 采用2个比特记录分支的历史，提高预测的准确度；
 - **研究表明：** 两位分支预测的性能与 n 位 ($n > 2$) 分支预测的性能差不多；【很神奇】
- 两位分支预测的状态转换如下所示：



基于2-bit饱和计数器的状态机

BHT的工作原理

- 分支历史表的步骤：

Step 1: 分支预测

- 当一条分支指令到达译码段（ID）的同时，依据从BHT读出的信息进行分支预测，取指。
- 若预测正确，就继续处理后续指令，流水线没有断流。
- 若预测错误，后续就要作废已经预取和分析，且进入流水线的指令，恢复现场，并从另一条分支路径重新取指令。

Step 2: 状态修改，更新BHT的状态。

- BHT只判定分支是否成功，还有一个需要解决的问题是：确定分支目标地址。

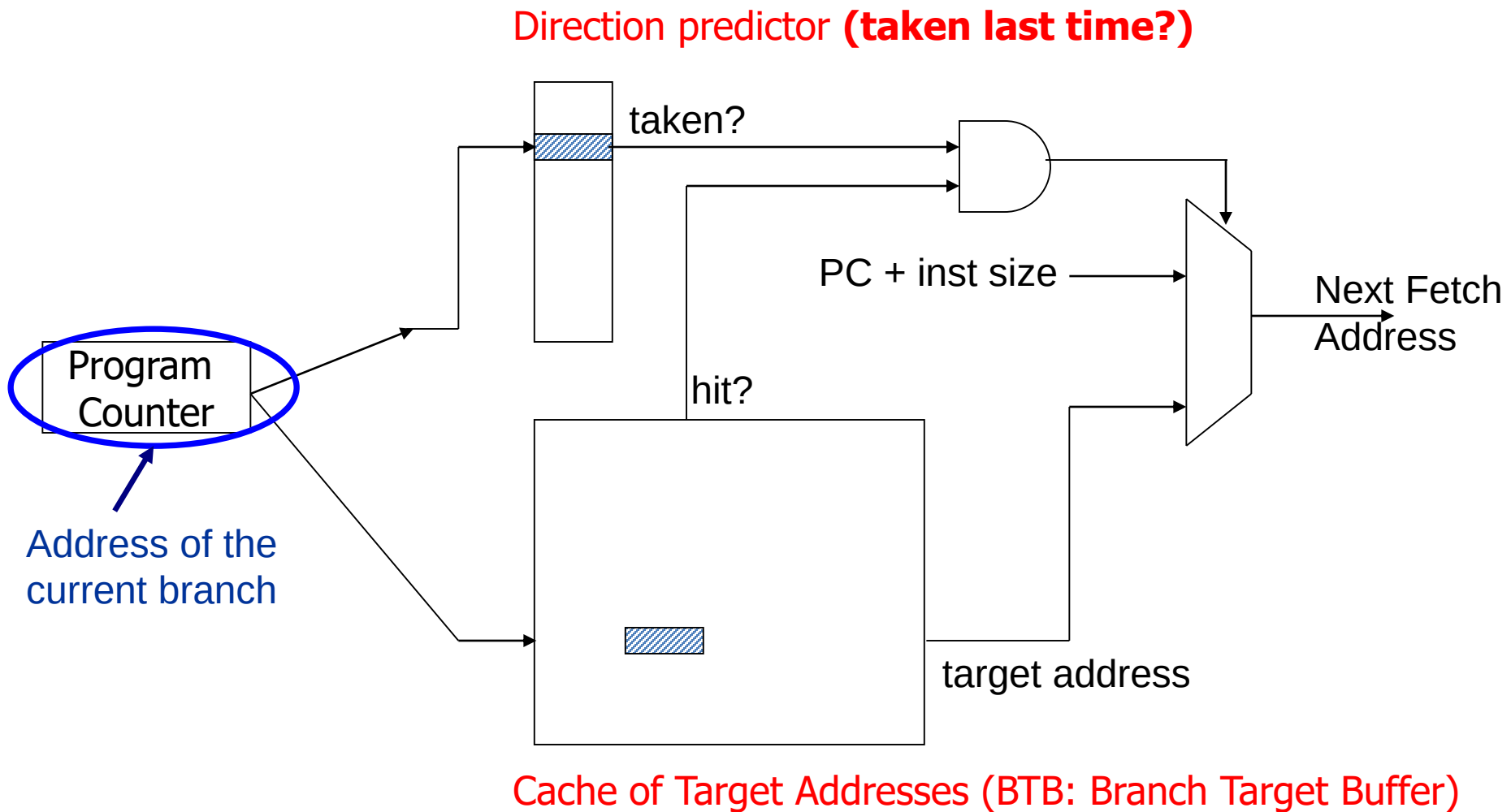
BHT的实现

- **研究表明：**对于SPEC89测试程序来说，具有大小为4K的BHT的预测准确率为82% ~ 99%。
 - 一般来说，采用4K的BHT就可以了。
- **问题：**对于当前的典型程序，4K BHT够好吗？
 - You can try with SimpleScalar simulator...
- BHT的实现方案：
 - BHT可以与分支指令一起存放在指令缓存*中
 - 也可以用一个专门的硬件结构来存储BHT

预测目标：分支目标缓冲器

- 目标：将分支成功时的开销降为 0
- 方法：利用分支目标缓冲器 (BTB)
 - 将**成功跳转**的分支指令的地址和它的跳转目标地址都放到一个缓冲区中保存起来，缓冲区以**分支指令的地址**作为标识。
 - 这个缓冲区就是分支目标缓冲器 (Branch-Target Buffer, 简记为BTB) 。
 - BTB可以解决动态分支预测的任务❶和❸：
 - 在BTB里面，就认为**是**分支指令； - task ❶
 - 在BTB里面，也能找到预测的目标地址。 - task ❸

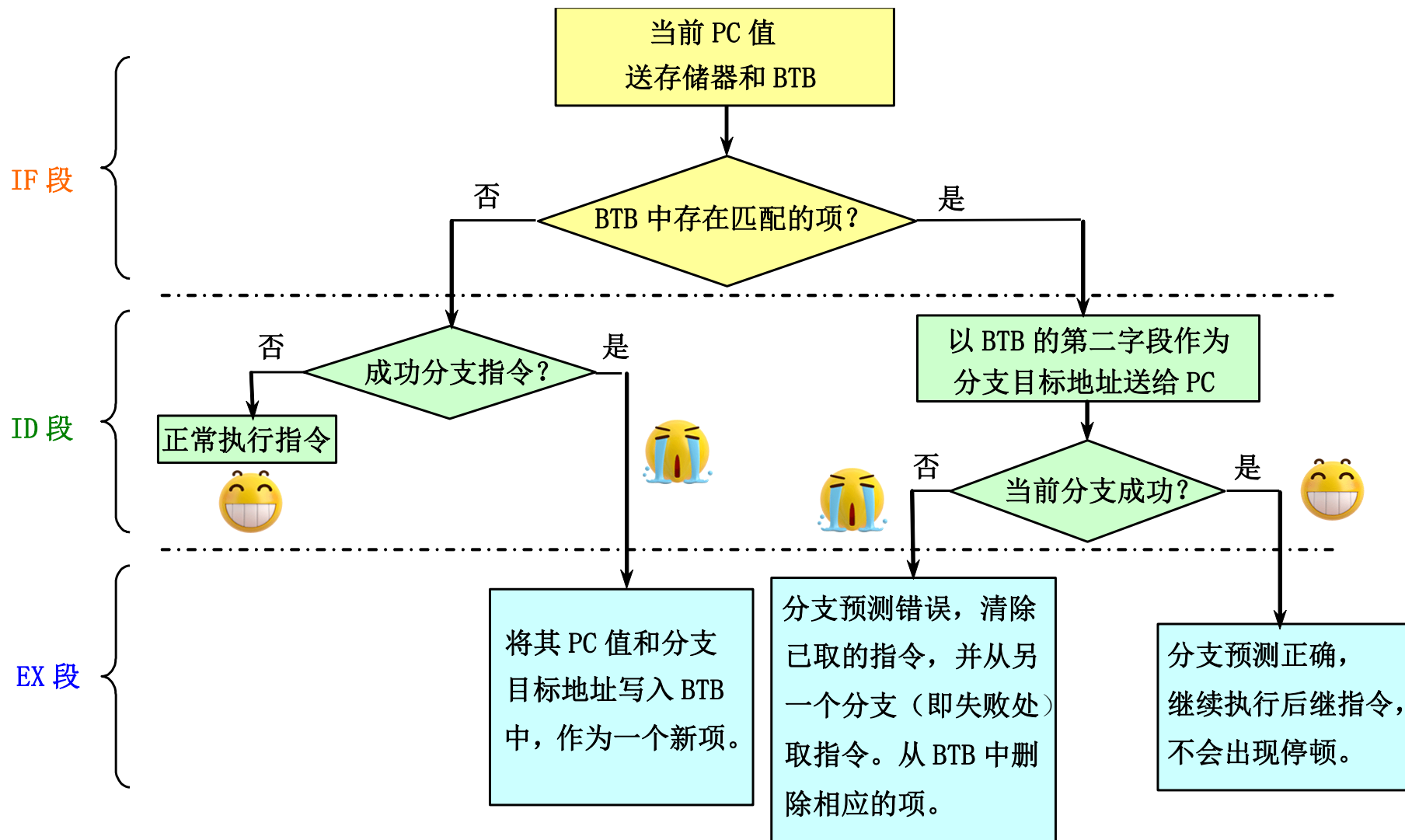
基于BTB的分支预测原理图



BTB的实现

- BTB可以用专门的硬件实现的一张表
- 表中的每一项通常有两个字段：
 - ① 执行过的成功分支指令的地址
 - 通常是**PC的低位比特**（没必要保存完整的PC）
 - 作为该表的匹配标识 (key)
 - ② 预测的分支目标地址

基于BTB的流水线关键流程



BTB机制下的延迟分析

指令在BTB中?	预测	实际情况	延迟周期
在	成功	成功	0
在	成功	不成功	2
不在	✖	成功	2
不在	✖	不成功	0

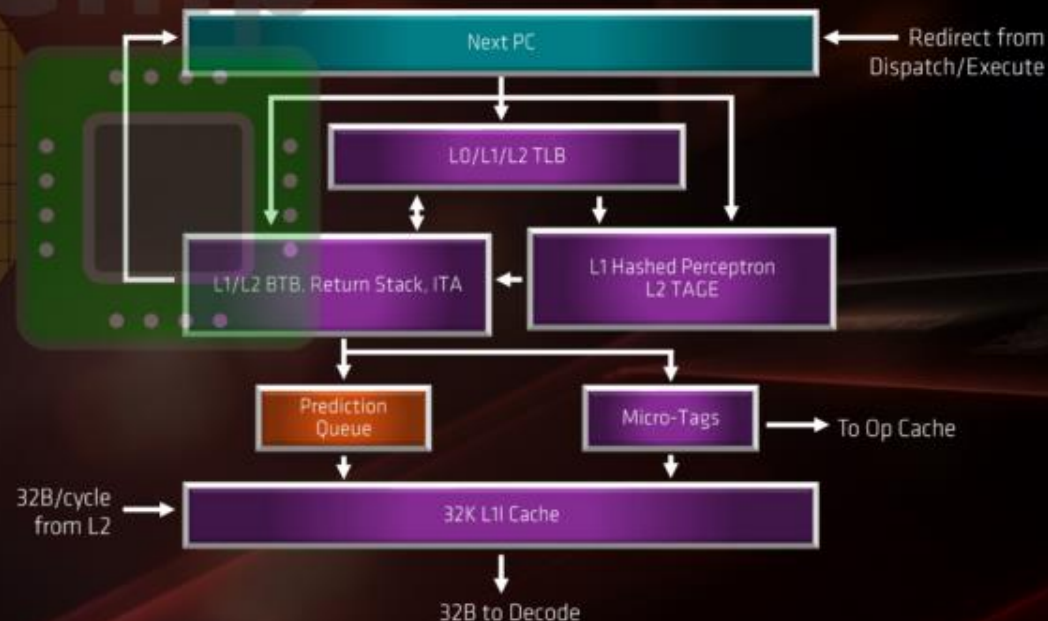
其它的分支预测方法

- 还有很多分支预测方法
 - Two-level Adaptive BP, 参考3.3节
 - Tournament BP, 参考3.3节

TAGE predictor

FETCH

- Improved branch prediction
- New TAGE branch predictor
- Larger BTBs
 - L0 BTB, 16 entries
 - L1 BTB, 512 entries (up from 256)
 - L2 BTB, 7K entries (up from 4K)
- Larger 1K indirect target array (ITA)
- ~30% lower mispredict rate target
- Optimized 32KB, 8-way L1I cache
 - Higher associativity
 - Improved prefetching
 - Improved utilization

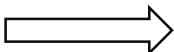


让指令飞起来：前瞻执行

- 前瞻执行 (speculative execution) 的基本思想：
 - 对分支指令的结果进行**猜测**，并假设这个**猜测总是对的**。然后，按这个**猜测结果（猜测的PC）**继续取指、**流出和执行后续的指令**。
 - 猜测执行的指令，其执行的结果不是写回到寄存器或存储器（Data Memory），而是放到一个称为**ROB（Re-Order Buffer）**的缓冲器中；
 - 等到相应的指令得到“确认”（commit）（即确实是应该执行的指令，猜测正确。）之后，才将结果从ROB中读出，并写入寄存器或存储器。

为什么写到ROB之中？

前瞻执行

- 基于硬件的前瞻执行结合了三种技术：
 - ① **动态分支预测**，用来确定后续要执行的指令；
 - ② 在是否是分支及其结果尚未确定之前，**前瞻地执行后续指令**；
 - ③ 用**动态调度**对基本块的各种组合进行跨块调度；
- 前瞻执行的实现：对**Tomasulo**算法加以扩充，就可以支持前瞻执行。
 - 把Tomasulo算法的写结果和指令完成加以区分，分成两个不同的段：
 - ① 写结果；  **为了精确异常以及后勤保障！**
 - ② 指令确认；

前瞻执行

- 写结果段

- 把前瞻执行的结果写到ROB中;
- 通过CDB在指令之间传送结果, 供需要用到这些结果的指令使用。

- 指令确认段

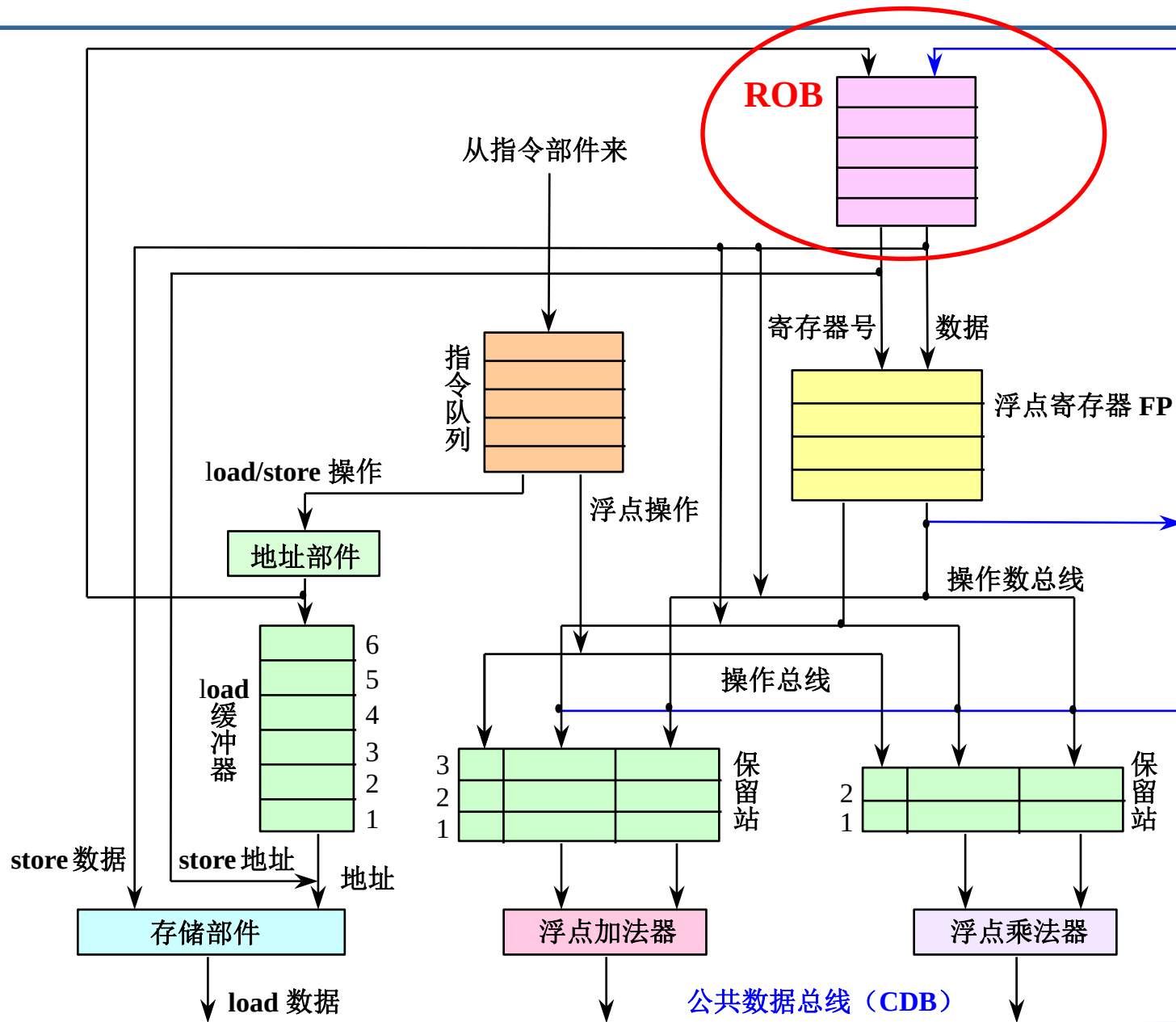
实现前瞻的关键思想:

允许指令乱序执行, 但必须**顺序确认**。

将结果写到寄存器或存储器;

- 如果发现前面的猜测是错误的, 那就不予以确认, 并从正确的分支路径开始重新取指执行;

支持前瞻执行的处理结构



前瞻执行的原理与实现

- ROB中的每一项由以下4个字段组成：
 - ① **指令类型**：标记该指令是分支指令、store指令或寄存器操作指令；
 - ② **目标地址**：给出指令执行结果应写入的**目标寄存器号**（如果是load或ALU指令）或 **存储器单元的地址**（如果是store指令）；
 - ③ **数据值字段**：用来保存指令前瞻执行的**结果**，直到指令得到确认；
 - ④ **就绪字段**：指出指令是否已经完成执行并且数据已就绪；

前瞻执行的原理与实现

- 在前瞻执行下，Tomasulo算法中保留站(RS)的换名功能是由ROB来完成的。
- 在前瞻执行机制中，指令的执行步骤：
 - ① 流出 (发射)
 - 从指令队列的头部取一条指令；
 - 如果有空闲的保留站 (设为r) **并且** 有空闲的ROB项 (设为b)，就流出该指令，并把相应的信息放入保留站r和ROB项b；
 - 如果保留站满 **或者** ROB满，就停止流出指令，直到它们都有空闲的项。 (**结构冲突**)

前瞻执行的原理与实现

② 执行

- 当两个操作数都就绪后，就可以执行该指令的操作；
- 如果有操作数尚未就绪，就等待，并不断地监测CDB。
(**RAW冲突**)

③ 写结果 【注意：与tomasulo原本的写结果不同】

- 当结果产生后，将该**结果**连同本指令在流出段所分配到的**ROB项的编号**放到CDB上，经CDB**写到ROB**以及所有等待该结果的保留站。【Data Forwarding】
- 释放产生该结果的保留站； **【保留站的使命已经完成】**
- Store指令的特殊处理： **【Store很特别，写的是MEMORY】**
 - 如果**要写入存储器的数据**已经就绪，就把该数据写入分配给该store指令的ROB项；
 - 否则，就监测CDB，直到那个数据在CDB上播送出来，这时才将之写入分配给该store指令的ROB项；

前瞻执行的原理与实现

④ 确认：不同类型指令的确认方式不同

- **除分支指令和store指令之外的其它指令**：当该指令到达ROB队列的头部而且其结果已经就绪时，就把该结果写入该指令的目标寄存器，并从ROB中删除该指令；
- Store指令：处理与上面类似，只是它把结果写入存储器；
- 分支指令：
 - 当预测错误的分支指令到达ROB队列的头部时，清空ROB，并从分支指令的另一个分支重新开始执行。（**错误的前瞻执行**）
 - 当预测正确的分支指令到达ROB队列的头部时，该指令执行完毕；

前瞻执行示例

例：假设浮点功能部件的延迟时间为：加法2个时钟周期，乘法10个时钟周期，除法40个时钟周期。对于下面的代码段，给出当指令MUL.D即将确认时的各个状态表内容。

L.D F6,34 (R2)

L.D F2,45 (R3)

MUL.D F0,F2,F4

SUB.D F8,F6,F2

DIV.D F10,F0,F6

ADD.D F6,F8,F2

前瞻执行示例

- 前瞻执行中MUL.D确认前，保留站和ROB的状态。

名称	保留站							
	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Add1	no							
Add2	no							
Add3	no							
Mult1	no	MUL	Mem[45+ Regs[R2]]	Regs[F4]			#3	
Mult2	yes	DIV		Mem[34+Regs[R2]]	#3		#5	

项号	ROB				
	Busy	指令	状态	目的	Value
1	no	L.D F6, 34 (R2)	确认	F6	Mem[34+Regs[R2]]
2	no	L.D F2, 45 (R3)	确认	F2	Mem[45+Regs[R3]]
3	yes	MUL.D F0, F2, F4	写结果	F0	#2 + #4 [F4]
4	yes	SUB.D F4, F2, F0	写结果	F4	#4 - #2 [F0]
5	yes	ADD.D F2, F0, F4	写结果	F2	#4 + #2
6	yes	ADD.D F0, F2, F4	写结果	F0	#4 + #2

请同学们自己对照Tomasulo调度算法和推测执行的过程，自己推导执行过程。

字段	浮点寄存器状态							
	F0	F2	F4	F6	F8	F10	...	F30
ROB项编号	3			6	4	5		
Busy	yes	no	no	yes	yes	yes	...	no

前瞻执行总结

- 前瞻执行的优缺点

- 主要优点:

- 通过ROB实现了指令的**顺序完成**;
 - 从而**能够**实现**精确异常**;
 - 很容易地推广到整数寄存器和整数功能单元上;

- 主要缺点:

- 复杂的控制导致所需的硬件太复杂;

本节提纲

- 指令级并行的概念
- 循环展开和指令调度
- 指令的动态调度
- 分支预测技术简介
- 多指令流出技术 — 自学!

